

Martinus-Gymnasium Linz am Rhein

Facharbeit

Sind die Hashing-Algorithmen MD5 und SHA-256 ihrem Alter gerecht?

**Moderne Passwortsicherheit anhand eines Authenti-
fizierungssystems**

David Wahab

Grundkurs Informatik

Herr Rohde

2025/26

Abgabetermin: 18.05.2026

Linz am Rhein

Kurzfassung

Die vorliegende Arbeit analysiert die Unterschiede zwischen den beiden Hashverfahren MD5 und SHA-256 und untersucht, welches der beiden Verfahren sein Alter gerechtfertigt hat. Dies wird anhand eines praktischen Tests in einem C++ Authentifizierungssystem getestet und anschließend ausgewertet. Die Ergebnisse zeigen, dass MD5 veraltet ist, während SHA-256 nur mithilfe zusätzlicher Schutzmaßnahmen noch als sicher gilt und heute als Standard angesehen wird.

Inhaltsverzeichnis

1	Einleitung	4
2	Theoretischer Teil.....	5
2.1	Grundlagen der Passwortsicherheit.....	5
2.2	Kryptographische Hashverfahren	7
2.2.1	Vergleich MD5 und SHA-256	8
2.3	Angriffsmethoden	10
2.3.1	Brute-Force	10
2.3.2	Rainbow-Tables	11
2.4	Erweiterte Schutzmechanismen.....	12
3	Methodischer Teil.....	13
3.1	Praktische Umsetzung in C++	13
3.1.1	Implementierung.....	13
3.1.2	Stresstest.....	14
3.2	Reflexion.....	14
4	Fazit und Ausblick.....	15
5	Literaturverzeichnis	16
6	Glossar.....	18
7	Anhang.....	20
8	Selbstständigkeitserklärung.....	22

1 Einleitung

“The human factor is truly security's weakest link” (Mitnick & Simon, 2001). Dieses Zitat verdeutlicht die alltägliche Situation, in der Nutzer regelmäßig auf schnelle Passwörter wie "123456" oder "Hallo123" zurückgreifen. Viele Nutzer sind der Meinung, das wäre ein sicheres Passwort. Doch trotz vieler Sicherheitswarnungen und Hinweise ignorieren die meisten diese. Tage später ist der Zugriff auf den Account verloren und man stellt sich die Frage, ob es die Schuld der Nutzer ist oder die des Unternehmens. Doch eins ist sicher, selbst wenn man ein starkes Passwort wie "7*Q,8\$Zw4Z>s" wählt, vertraut man dem Unternehmen hinter dem Bildschirm, dass das Passwort sicher aufbewahrt wird und mit diesem verantwortungsbewusst umgegangen wird.

Vor diesem Hintergrund wird die Bedeutung der Passwortsicherheit immer wichtiger. Durch die steigende Rechenleistung von Standardcomputern nehmen die gezielten Angriffe auf Unternehmen drastisch zu. Um dagegen anzukämpfen, müssen Unternehmen immer modernere kryptographische Verfahren zur sicheren Passwortspeicherung einsetzen. Die bekanntesten kryptographischen Hashverfahren sind MD5 und SHA-256.

Daher untersucht die vorliegende Arbeit folgende Fragestellung: „Sind die Hashing-Algorithmen MD5 und SHA-256 ihrem Alter gerecht?“

Um diese Frage zu beantworten lässt sich hierzu die Arbeit in einen theoretischen und einen methodischen Teil gliedern. Der theoretische Teil fokussiert sich darauf, was ein sicheres Passwort überhaupt ausmacht (Kapitel 2.1). Zunächst wird der Unterschied zwischen MD5 und SHA-256 Algorithmen untersucht (Kapitel 2.2). Daran anschließend folgen erweiterte Schutzmaßnahmen neben den Algorithmen selbst (Kapitel 2.4). Anschließend werden die Methoden im Methodischen Teil getestet und analysiert. Zudem erfolgt die Auswertung der Testergebnisse (Kapitel 3) und eine darauffolgende Reflexion (Kapitel 3.2). Abschließend erfolgt das Fazit (Kapitel 4).

2 Theoretischer Teil

2.1 Grundlagen der Passwortsicherheit

In der Informatik wird die Stärke eines Passworts nicht anhand der Anzahl von Sonderzeichen, sondern anhand der Entropie gemessen. Entropie kann viele Definitionen haben, aber in der Informatik ist diese fest definiert. So hat Claude Elwood Shannon (1948) die Entropie als mathematisches Maß für Überraschung und Unsicherheit einer Nachrichtenquelle definiert und entwickelte eine mathematische Definition:

$$H = - \sum_{i=1}^n p_i \log_2(p_i)$$

In der Formel steht p_i für die Wahrscheinlichkeit, dass ein bestimmtes Ereignis auftritt. Die Entropie selbst hat Shannon als H definiert. Man berechnet die Wahrscheinlichkeit für jedes mögliche Ereignis, multipliziert diese mit dem Logarithmus der Wahrscheinlichkeit p_i und rechnet am Ende alles zusammen.

Sehen wir uns ein Beispiel an:

1. PASSWORT

2. SAPRTSOW

Welches der beiden Klartexte enthält mehr Informationen? Wahrscheinlich würden die meisten intuitiv auf das Erste tippen, das ist jedoch falsch. Die Entropie des 1. „PASSWORT“ ist niedriger, da es vorhersehbarer ist. Wer die Zeichenfolge PASS liest, kann sich die restlichen Buchstaben schlussfolgern, nämlich WORT. Beim 2. hingegen kann man unmöglich vorhersehen, dass der Klartext "SAPRTSOW" lauten wird, somit ist die Entropie sehr hoch und es gibt „Überraschungen“, da nach jedem Buchstaben eine neue Information geliefert wird. Im Kontext der Passwortsicherheit ist die Entropie ein gutes Maß dafür, wie stark ein Passwort gegen bloße Brute-Force Attacken resistent ist. Die Entropie wird in Bits angegeben. Je mehr Bits, desto höher ist die Entropie und umso stärker ist das Passwort. Viele denken, dass ein kompliziertes Passwort vor Angreifern sicherer wäre, zum Beispiel: „73@\$5w“ (BSI, 2023). Sowas kann man sich

jedoch nicht merken und zudem ist es nicht sicher. Dahingegen ist „TestHalloLanglStSicherer“ deutlich sicherer, da es eine höhere Entropie aufweist. Mit folgender Formel lässt sich die theoretische Entropie eines Passworts vereinfachend abschätzen, wenn man definiert, dass alle Zeichen (Ereignisse) die gleiche Wahrscheinlichkeit besitzen.

Daraus folgt:

$$H = n * \log_2(s)$$

Dadurch lässt sich die Entropie eines Passworts einfacher messen (TechTarget, 2023). Es gibt 96 verschiedene Zeichen (QWERTY + €), wenn man alle Buchstaben, Zahlen und Sonderzeichen nutzt. Also ist $s = 96$ und n ist die Länge des Passworts. „73@\$5w“ hat 6 Zeichen, das heißt: $H = 6 * \log_2(96) = 39,509$. Daraus folgt, dass „73@\$5w“ eine Entropie (Bits) von ≈ 40 aufweist und somit ein „schwaches Passwort“ ist (Specops Software, 2025). „TestHalloLanglStSicherer“ hingegen ist viel einfacher zu merken und zudem sicherer, da die Entropie hier 158,0391 (Bits) aufweist und somit ein „sehr gutes Passwort“ ist (Specops Software, 2025). In diesem Zusammenhang muss jedoch hinzugefügt werden, dass es sich hierbei um die theoretisch maximale Entropie handelt und die tatsächliche Entropie geringer ausfällt, da das Passwort aus realen Wörtern besteht. Mithilfe der vereinfachten Formel kann man immer die theoretische Stärke seines eigenen Passworts messen. Nachdem man sein Passwort ausgewählt hat, kommen Firmen ins Spiel, die damit sicher umgehen, und hier kommt der erste Mythos: Firmen verschlüsseln keine Passwörter, sondern hashen sie (OWASP, o. D.).

Doch was ist Hashen und was ist Verschlüsseln? Der Unterschied zwischen Hashing und Verschlüsselung lässt sich wie folgt erklären: Hashen ist wie eine mathematische Einbahnstraße. Man gibt etwas ein und ein mathematischer Algorithmus „hasht“ es in eine daraus folgende Zeichenkette. Egal wie und womit man es versucht, kann man es nicht zurückkehren (Hughes, 2022). Wenn man „Passwort“ mithilfe eines beliebigen Hash-Algorithmus hasht, kommt am Ende ein Hash raus, der immer wie eine zufällige Buchstaben- und Zeichenkette aussieht. Dabei entscheidet der ausgewählte Algorithmus, wie lang die Kette ist und wie schnell der eingegebene Klartext im Falle „Passwort“ gehasht wird. Damit ein Hashing Algorithmus als „sicher“ gilt, muss er die folgenden 3 Funktionen erfüllen (Paar & Pelzl, 2010, S. 296):

Die erste Funktion heißt Erste Urbildresistenz („Preimage Resistance“). Das bedeutet, dass es rechnerisch unmöglich ist, aus einem gegebenen Hashwert y den ursprünglichen Klartext x (Urbild) zu berechnen. Gegeben sind die Hashfunktion H und ein Hashwert y . Es darf nicht möglich sein, die Eingabe x zu finden: $H(x) = y$ (Paar & Pelzl, 2010, S. 296).

Dann gibt es die zweite Urbildresistenz („Second Preimage Resistance“). Die soll gewährleisten, dass es unmöglich ist, zu einem gegebenen Klartext x , einen zweiten abweichenden Klartext zu finden x' , der denselben Hashwert erzeugt. Es gilt: $x \neq x'$ und $H(x) = H(x')$ (Paar & Pelzl, 2010, S. 296).

Die dritte und letzte Funktion eines Hash-Algorithmus ist die Kollisionsresistenz („Collision Resistance“). Das ist die schwierigste Eigenschaft eines Hash-Algorithmus und einer der Gründe, weshalb MD5 scheiterte. Im Unterschied zur zweiten Urbildresistenz, sorgt die dritte dafür, dass es einem Angreifer nicht möglich ist, selbstständig zwei unterschiedliche Klartextpaare mit demselben Hashwert zu finden. Es gilt: $H(x) = H(x')$ (Paar & Pelzl, 2010, S. 296, 299).

Bei einer Verschlüsselung verläuft es wiederum ganz anders. Wenn man einen Klartext eingibt, durchläuft dieser verschiedene mathematische Algorithmen. Doch der Unterschied liegt darin, dass man es immer mithilfe eines Schlüssels umkehren kann (Schlüssel-Schloss-Prinzip) (Paar & Pelzl, 2010, S.150, 296). Doch warum sollte man Passwörter nicht verschlüsseln? Es ist einfach, weil ein Angreifer, sobald er Zugriff auf die Verschlüsselung hat, sie jederzeit entschlüsseln kann. Mit heutigen technischen Mitteln und Dokumentationen ist das einfacher denn je.

2.2 Kryptographische Hashverfahren

Wie bereits im vorherigen Kapitel besprochen, sind Hashes eine mathematische Einbahnstraße und nicht rückgängig zu machen. Ein großes Problem bei Hashes sind sogenannte „Kollisionen“. Ein berühmter Hash-Algorithmus, der lange als unantastbar galt, war der MD5-Algorithmus. Doch dieser wurde von Wang & Yu (2005) aufgrund von Kollisionsangriffen als unsicher deklariert. Durch den Befund von Kollisionen wurden zudem viele andere Hash-Algorithmen zurückgerufen und es mussten neue Lösungen wie die SHA-2-Familie, darunter den SHA-256-Algorithmus, her. Bis heute gilt

der SHA-256 als sicherer Hash-Algorithmus und wird von den meisten als Industriestandard verwendet.

2.2.1 Vergleich MD5 und SHA-256

Der MD5 (Message-Digest Algorithm 5) wurde von Ronald Rivest (1992) entwickelt und veröffentlicht. Er gilt als ein sehr schneller Algorithmus. Der Hashwert des MD5 hat eine Länge von 128 Bit, was in hexadezimaler Darstellung einer 32-stelligen Zeichenkette entspricht. Aufgrund seiner hohen Kollisionsanfälligkeit gilt er als kryptografisch gebrochen. Im Gegensatz dazu steht der SHA-256 Algorithmus, der von der NIST (National Institute of Standards and Technology) 2001 entwickelt und herausgegeben wurde. Der Algorithmus gehört der SHA-2 Familie an. Ein deutlicher Unterschied, der von außen direkt zu erkennen ist, ist die Länge des Hashes, denn es ist ein 256-Bit Algorithmus und weist somit eine 64-stellige Hexadezimallänge auf. Das Problem ist, dass er viel langsamer ist als der MD5-Algorithmus, was sich später jedoch als Vorteil erwiesen hat.

Um die Unterschiede zwischen den beiden Algorithmen genauer zu erkennen, ist eine detailliertere Betrachtung notwendig. Den MD5-Algorithmus kann man in einen 5-stufigen Prozess unterteilen: Man verwendet erneut das Beispiel mit dem „Passwort“. Zunächst wird das Passwort so erweitert „gepadet“, dass die Bits kongruent zu $448 \bmod 512$ sind. Im Beispiel „Passwort“ hat das Passwort 8 Zeichen und ist in der UTF-8 Kodierung 8 Bytes groß: $8 * 8 = 64$ Bit. Anschließend wird überprüft, wie viele Bits noch fehlen, bis es kongruent zu $448 \bmod 512$ ist: $448 - 64 = 384$ Bits fehlen. Das heißt: Es wird ein „1“ Bit an den 64-Bits und 383 weitere „0“ angehängt.

Im zweiten Schritt wird die vollständige Länge an das Padding angehängt. Das bedeutet, man nimmt die 64-Bit Darstellung des Klartexts „Passwort“ und fügt sie an die 448 Bits an, sodass die Gesamtlänge ein Vielfaches von 512 Bits ergibt.

Im dritten Schritt wird der Klartext für den Algorithmus optimiert, indem vier Puffer (Register) jeweils mit 32 Bit initialisiert werden, sodass am Ende zusammen ein Hashwert von 128 Bit erhalten wird. Diese Puffer enthalten fest definierte Startwerte. In den zuvor initialisierten Puffern A, B, C und D wird das „Passwort“ eingefügt und in seiner Hexadezimaldarstellung angegeben.

Im vierten Schritt werden die Werte A, B, C und D in AA, BB, CC und DD kopiert, um die neuen Werte mit den ursprünglichen zu vermischen. Jetzt werden 4 sogenannte „Hilfsfunktionen“ hinzugefügt:

$$F(X,Y,Z) = (X \text{ and } Y) + (\text{not } X \text{ and } Z)$$

$$G(X,Y,Z) = (X \text{ and } Z) + (Y \text{ and not } Z)$$

$$H(X,Y,Z) = (X \oplus Y \oplus Z)$$

$$I(X,Y,Z) = (Y \oplus (X \text{ or not } Z))$$

Sinn und Zweck dieser Funktionen ist es, am Ende eine komplett durchmischte Bit-ebene zu erhalten. Durch Bit Operationen wie „and“, „XOR“ und „not“ werden die Daten auf hochkomplexe Weise in den 64 Rechenrunden verändert und Manipuliert. Selbst die kleinste Änderung am Klartext sorgt dafür, dass ein komplett unterschiedlicher Hash erzeugt wird, was einen der wichtigsten Schritte von Hashfunktionen bildet.

Im fünften und letzten Schritt des Algorithmus werden alle Puffer zusammengeführt und nach dem „low order byte of A“ sortiert. Er beginnt mit dem kleinsten Teil A und arbeitet sich bis zu D vor. Am Ende entsteht ein 128-Bit Wert, der in Hexadezimalzahlen in eine 32 Zeichen lange Kette umgewandelt wird: „3e45af4ca27ea2b03fc6183af40ea112“. Die gesamte Funktionsweise des MD5 basiert auf der Originalpublikation von Rivest (1992).

Obwohl der Algorithmus lange Zeit als sicher galt, weisen seine internen Operationen kritische und strukturelle Schwächen auf. Diese Schwachstellen führen zu sogenannten Kollisionen. Die Schwächen des Algorithmus sind auf die internen Rundenfunktionen zurückzuführen. So haben Wang und Yu (2005) bewiesen, wie anfällig der Algorithmus für Kollisionen ist. Sie beweisen das Ganze, indem sie durch gezielte Manipulationen in den 64 Rechenrunden (Differential Attacks) denselben Hashwert aus zwei verschiedenen Eingabewerten erzeugen können.

An dieser Stelle setzt die SHA-2 Familie an, die so konzipiert wurde, dass es fast unmöglich ist, Kollisionen zu erzeugen oder auftreten zu lassen. Zum einen durch die höhere Bitrate, zum anderen durch die höhere Komplexität. Der Algorithmus der SHA-2 Familie ist ähnlich wie der MD5-Algorithmus, unterscheidet sich aber an einzelnen Stellen sehr stark. So hat der SHA-256-Algorithmus anstatt 4 Puffer (A,B,C und D) jetzt 8 (A bis H) (NIST, 2015, S. 21). Ein weiterer und entscheidender Unterschied zu MD5

ist das sogenannte „Message Scheduling“, bei dem im Gegensatz zu MD5 das Passwort von seinen 512 Bits auf 64 Wörter aufgeteilt wird, anstatt unverändert zu lassen (NIST, 2015, S. 21). Somit ist das der entscheidende Punkt gegen Kollisionsangriffe. Neben dem Message Scheduling gibt es zusätzlich komplexere interne Rechenoperationen. Hierbei werden 6 Funktionen verwendet. Die ersten beiden lauten *Ch* (Choose) und *Maj* (Majority), sowie zusätzlich vier Sigma Funktionen ($\sum_0 \sum_1 \sigma_0 \sigma_1$). Hier ist es wichtig zu erwähnen, dass die Sigma Funktionen anders agieren als in der Mathematik, denn sie stehen für komplexe Bit Rotationen und Kombinationen wie ROTR und XOR ($\oplus$) (Cryptrec, 2015, S. 8; NIST, 2015, S. 10). Durch diese komplexen und asymmetrischen Verschiebungen von Bits entsteht bei der noch so kleinsten Änderung ein komplett unterschiedliches Ergebnis, das die Kollisionsresistenz, also den dritten Punkt von Hash-Algorithmen, erfolgreich erfüllt.

2.3 Angriffsmethoden

Wie in der Informatik üblich, gibt es Wege, sich Zugang zum System zu verschaffen. Dabei werden verschiedene Angriffsmethoden eingesetzt, in dieser Arbeit werden die beiden wichtigsten besprochen. Einmal die Rainbow-Tables und die berühmteste und bekannteste „Brute-Force Attacke“.

2.3.1 Brute-Force

Brute Force kommt aus dem Englischen und bedeutet „rohe Gewalt“. Es beruht darauf, systematisch etwas auszuprobieren. In der Informatik ist es dafür bekannt, dass man mit bloßem Probieren Passwortkombinationen erraten und knacken kann. Es beginnt mit dem ersten Buchstaben des Alphabets oder der von Ihnen festdefinierten Zeichenkette, zum Beispiel „A“. Anschließend iteriert der Algorithmus das gesamte Alphabet jeweils ein Zeichen nach dem anderen: zuerst „A“, dann „B“, dann „C“ usw., bis er bei „Z“ angekommen ist (Paar & Pelzl, 2010, S. 7, 8). Doch sofort wird ein Problem sichtbar. Die Anzahl der möglichen Buchstabenkombinationen steigt exponentiell. Bei einem Alphabet mit 26 Buchstaben gibt es für eine Zeichenkette nur 26 Möglichkeiten. Für die zweite Zeichenkette sind es bereits $26^2 = 676$ Kombinationen. Für 3 Zeichenketten „AAA“ liegt die Zahl noch höher, nämlich $26^3 = 17576$. Nutzt man jedoch den vollen Zeichensatz, also 96 Zeichen, so ist man ganz schnell bei 96^3 .

In der Praxis verläuft es dennoch anders. Denn hier wird das Passwort nicht einfach so geraten, sondern durchläuft bei jeder einzelnen Berechnung viele kleine Schritte. In der Regel wird zunächst der Buchstabe iteriert, also „AA“ usw. Für jeden String wird dieser einmal mit dem gesuchten Hash gehasht und anschließend mit dem Gesuchten verglichen. Doch wenn man selbst ein Passwort ausprobiert, fällt einem auf, dass es sehr zeitaufwendig ist, da der Computer jeden einzelnen String einzeln ausrechnet und diesen kombiniert. Bei kurzen, knappen Passwörtern fällt das nicht auf, aber je länger ein Passwort wird, desto länger braucht der Algorithmus, diesen zu knacken. So wurde für diese Facharbeit ein Brute-Force Algorithmus entwickelt, der direkt beweist, wie effizient und zugleich ineffizient Brute-Force sein kann (siehe Tabelle 1 im Anhang). Zudem sind die Ergebnisse stark vom Hash Algorithmus abhängig und bei Passwörtern, die mit dem MD5 Algorithmus gehasht wurden, kam man immer am schnellsten zum Zielhash, während es beim SHA-256 Algorithmus deutlich länger dauerte.

Doch woran liegt es, wenn MD5 und SHA-256 beides Hashalgorithmen sind und sich nur anhand der Architektur unterscheiden? Genau hier liegt das Problem: Während der MD5 Algorithmus relativ leicht verständlich ist, ist der SHA-256 Algorithmus deutlich komplizierter und komplexer. Der Computer benötigt mehr Zeit, um einen einzelnen Hash zu berechnen und in der heutigen Zeit, in der praktisch jeder eine gute CPU besitzt, ist MD5 neben den Kollisionen nicht komplex genug, um Zeit rauszuzögern, während es beim SHA-256 komplett anders aussieht. Die Brute-Force Zeit steigt exponentiell an, je mehr Buchstaben im Spiel sind, wie bereits oben mathematisch bewiesen wurde.

2.3.2 Rainbow-Tables

Um das Zeithindernis zu umgehen, greifen Hacker auf „dictionary attacks“ (Wörterbuchangriffe) zurück. Es gibt viele bekannte öffentliche Listen, die kompromittierte Passwörter enthalten, darunter „rockyou.txt“, die auch in dieser Arbeit verwendet wurde. Der Unterschied zu klassischen Brute-Force besteht darin, dass nicht jede mögliche Zeichenkombination durchprobiert wird. Stattdessen wird eine vorgefertigte Liste (rockyou.txt) abgearbeitet, was viel Zeit einspart.

Jedoch gibt es eine effizientere Methode, nämlich die Rainbow-Tables. Diese funktionieren ähnlich wie Wörterbuchangriffe, bieten jedoch zusätzliche Funktionen an. Rainbow-Tables enthalten neben den meistgenutzten Passwörtern vorberechnete

Hashketten und spezielle Reduktionsfunktionen, um eine verbesserte Rechenzeit und einen geringeren Speicherplatzverbrauch zu ermöglichen (Oechslin, 2003).

Um Zeit und Komplexität zu vermeiden, wurde in dieser Arbeit die Wörterbuchliste mit der Rainbow-Table kombiniert. Zuerst läuft die Datei „rockyou.txt“ durch, anschließend greift das Programm bei keinem Erfolg auf die Rainbow-Table-API (MD5Decrypt, o.D.) zu. In Tabelle 2 (im Anhang) wird der zeitliche Vorteil von Rainbow-Tables gegenüber Brute-Force verdeutlicht. Dort fällt auf, dass Rainbow-Tables bei einfachen Passwörtern viel schneller und effektiver abschneiden als bei Brute-Force Attacken.

2.4 Erweiterte Schutzmechanismen

In der Informatik, insbesondere in der Cybersecurity, herrscht ein ständiges Wettrüsten zwischen Angreifern und Verteidigern. Da einfache Hashes in den vorherigen Kapiteln bekanntermaßen anfällig für Rainbow-Tables und Brute-Force sind, müssen zusätzliche Sicherheitsmaßnahmen her.

Darunter fällt das sogenannte „Salting“. Der Salt funktioniert folgendermaßen: Wählt der Nutzer ein Passwort wie „123456“, wird es durch einen individuellen Salt „@3&fS%“ erweitert, und das Ergebnis würde folgendermaßen aussehen: „123456@3&fS%“. Durch den Salt fallen sämtliche Wörterbuchangriffe und Rainbow-Tables weg, da das Passwort unerkennbar ist und in der Datenbank nicht mehr auffindbar ist (OWASP, o.D.). Den Angreifern bleibt somit das Brute-Force als letztes Mittel. Zwar erschwert der Salt das Vorgehen massiv, verhindert es jedoch nicht zu hundert Prozent, weshalb weitere Schutzmaßnahmen erforderlich sind. Darunter ist der Pepper, der neben dem Salt komplett anders funktioniert.

Der Pepper (Pfeffer) ist fest im Code des Unternehmens versteckt und bleibt geheim sowie unverändert (OWASP, o.D.). Das heißt, für das gewählte Passwort werden zusätzlich Salt und Pepper hinzugefügt. Das Passwort wird somit aus *Klartext* + *salt* + *pepper* = „123456@3&fS%p3p3r_|37_g3h3|m“ gebildet. Sobald ein Angreifer Zugriff auf die Datenbank verschafft hat, würde die benötigte Zeit zum knacken des Passworts, astronomischen Maße annehmen, da die Anzahl der möglichen Zeichenkombinationen zu hoch ist.

Daher verlagern Angreifer ihren Fokus, um an Passwörter zu gelangen, auf den Menschen selbst, der die größte Schwachstelle in der Cybersicherheit ist (Mitnick & Simon,

2001). Beispielsweise durch Phishing oder Social Engineering. Um diese Schwachstelle zu umgehen, wurde die Zwei Faktor Authentifizierung (2FA) eingeführt, die inzwischen als Industriestandard anerkannt ist (CISA, o.D.). Sobald ein Angreifer ein Passwort erfolgreich kompromittiert, braucht der Angreifer ein zweites physisches Gerät, das nur der Besitzer des Passworts besitzt, um den Login zu autorisieren (CISA, o.D.).

3 Methodischer Teil

3.1 Praktische Umsetzung in C++

3.1.1 Implementierung

Um die Unterschiede zwischen MD5 und SHA-256 Hashes praktisch besser zu verstehen, wurde ein Authentifizierungssystem in C++ entwickelt. Um den Erkenntnisweg nachvollziehbar zu gestalten, wurde die Implementierung in sechs Versionen unterteilt, von der unsicheren Methode der Passwortspeicherung bis hin zu einer der sichersten Methoden, iterativ entwickelt (V1-V6).

- Version 1 (main_v1_unsicher.h): Das Passwort wurde in Klartext in v1_user.txt gespeichert, was bei einem Datenleak zur sofortigen Kompromittierung führt.
- Version 2 (main_v2_hashing_MD5.h): Einführung des MD5 Hashalgorithmus. In v2_user.txt befindet sich ein kryptografischer Text, der unlesbar ist.
- Version 3 (main_v3_hashing_SHA256.h): Hash Algorithmus von MD5 auf SHA-256 aktualisiert.
- Version 4 & 5 (main_v4_saltied_hashing.h & main_v5_peppered_hashing.h): Implementierung erweiterter Schutzmaßnahmen gegen Rainbow-Tables und Brute-Force Angriffe. Einführung von Secure Memory Zeroing.
- Version 6 (main_v6_2fa.h): Einführung der 2FA zum Schutz vor Phishing.

Zudem wurden verschiedenste forensische Werkzeuge bereitgestellt, um das System zu stresstesten. Darunter: Brute-Force Programm unter main_brute_force.h, ein Rainbow-Table-Angriff, der mithilfe einer API läuft (main_rainbow_table.h), sowie ein Tool, das all die genannten Angriffe umfasst und automatisch erkennt, ob der gesuchte Hash MD5 oder SHA-256 ist. Ebenso wurden zur Arbeit passend ein Entropietester zur

Bewertung der Passwortkomplexität und ein Hashgenerator zur schnellen Erstellung von Hashes eingebaut.

3.1.2 Stresstest

Um die Unterschiede und Stärken zwischen MD5 und SHA-256 zu testen, wurden mithilfe von Brute Force in „main_brute_force.h“ mehrere Tests durchgeführt, indem über den Hash Generator MD5 und SHA-256 Hashes ohne Salt generiert und anschließend getestet wurden. Abbildung 1 (im Anhang) veranschaulicht, wie viel länger ein Computer für SHA-256 als für MD5 benötigt. Dies lässt sich auf die höhere architektonische Komplexität des SHA-256 Algorithmus zurückführen, der, wie in Kapitel 2.2 beschrieben, deutlich mehr Rechenoperationen pro Hash benötigt. Die Schwerfälligkeit erweist sich hier als Sicherheitsvorteil, da es für den Angreifer länger dauert, ein Passwort zu knacken. Zudem belegt die Abbildung den exponentiellen Anstieg der Rechenzeit mit zunehmender Zeichenzahl. Während kurze Passwörter von ein bis drei Zeichen in Bruchteilen geknackt werden, benötigt es bei fünf bis sechs Zeichen deutlich mehr Zeit.

Ein weiterer Schwerpunkt des Tests lag auf Wörterbuchangriffen mit Rainbow-Tables. Tabelle 3 (im Anhang) zeigt, dass einfache Passwörter wie „test“ ohne zusätzlichen Schutz in Sekunden geknackt werden können, sowohl bei MD5 als auch bei SHA-256. Wird das Passwort jedoch mit Salt und Pepper versehen, liefert der Rainbow-Table keine Ergebnisse mehr. Der Salt sorgt dafür, dass der Hash in keiner vorgefertigten Datenbank zu finden ist, sodass die Passwörter unabhängig von ihrer Komplexität unerkannt bleiben. Für den Angreifer bleibt nur noch der Brute-Force-Angriff, der bei gesalteten und gepepperten Hashes kaum Aussicht auf Erfolg hat, da die benötigte Zeit zum Knacken astronomisch groß ist. Ein Angriff durch reinen Brute-Force gilt als unrealistisch und für den Menschen nicht machbar.

3.2 Reflexion

Rückblickend auf den Entstehungsprozess lassen sich sowohl Stärken als auch Schwächen erkennen. Was gut lief, waren die schrittweise Entwicklung aller einzelnen

Versionen sowie die restlichen Tools, jedoch mit ein paar Einwänden, denn in der Realität funktionieren die meisten genannten Dinge anders als zum Beispiel in v3.users.txt. In der Realität wird das Passwort nicht einfach in einer .txt Datei gespeichert, sondern auf einem Server, auf den ein Angreifer nicht so einfach Zugriff hat. Zudem ist der Brute-Force Algorithmus extrem vereinfacht dargestellt. Ein Hacker würde den Brute-Force Algorithmus nicht nur allein auf einer CPU laufen lassen, sondern auch auf GPUs, in denen die Ergebnisse beim Stresstest komplett anders aussehen würden und Milliarden statt einer Million Passwörter durchiterieren könnten. In einer erweiterten Version würde sich der Brute-Force Algorithmus, wie oben genannt, stark verändern, entweder durch GPUs oder durch Threads aus den CPUs. Zudem würde sich eine Implementierung eines alternativen Hashing Algorithmus anbieten, wie den bcrypt, der darauf spezialisiert wurde, gegen Brute-Force anzukämpfen.

4 Fazit und Ausblick

Die vorliegende Arbeit hat gezeigt, dass MD5 und SHA-256 trotz vieler Gemeinsamkeiten völlig unterschiedlich sind und unterschiedliche Funktionen besitzen. Während MD5 als kryptografisch kompromittiert gilt, wurde SHA-256 lange Zeit als zuverlässig eingeschätzt. Doch mit der steigenden Rechenleistung moderner Hardware reicht SHA-256 allein nicht mehr aus. Daher sind zusätzliche Sicherheitsmaßnahmen wie Salt und Pepper bei SHA-256 notwendig, um mit der zunehmenden Rechenleistung moderner Computer mitzuhalten, wie die Authentifizierungsprogramme Version 4 und 5 zeigen. Um die Leitfrage zu beantworten, lässt sich sagen, dass MD5 seinem Alter nicht mehr gerecht wird, während SHA-256 nur in Kombination mit modernen Schutzmaßnahmen weiterhin als sicher gilt. Ein Blick in die Zukunft zeigt, dass sich Hashing Algorithmen und Passwörter wandeln werden. Die Bedrohung durch Quantencomputer wächst, sodass SHA-256 mithilfe des Grover-Algorithmus die effektive Sicherheit von 256 auf 128 Bit reduziert wird. Deshalb arbeitet man bereits an einem Post-Quantum Standard, inklusive SHA-512. Menschliche Schwachstellen werden durch biometrische Schlüssel (Passkeys) ersetzt, die traditionelle Passwörter ablösen. Zudem gewinnt künstliche Intelligenz zunehmend an Bedeutung, da sie Zero-Day-Exploits leichter erkennen, Login-Systeme umgehen und damit sensible Daten gefährden können. Das Wettrüsten zwischen Angreifern und Verteidigern setzt sich somit fort.

5 Literaturverzeichnis

- Bundesamt für Sicherheit in der Informationstechnik & Bundeskanzleramt, Referat 612. (2023). *Untersuchungen zum Passwortverhalten*. Bundesregierung. <https://www.bundesregierung.de/re-source/blob/975272/2258498/64d1897d8417a7081f5ee5262ea1b83f/de-password-download-bericht-data.pdf>
- CRYPTREC. (2015). *Security Evaluation of SHA-224, SHA-512/224, and SHA-512/256*. <https://www.cryptrec.go.jp/exreport/cryptrec-ex-2401-2014.pdf>
- Hughes, A. (2022, 1. März). *Verschlüsselung vs. Hashing vs. Salting - Wo liegen die Unterschiede?* Ping Identity. <https://www.pingidentity.com/de/resources/blog/post/encryption-vs-hashing-vs-salting.html>
- MD5Decrypt. (o. D.). *MD5 Decrypt API*. Abgerufen am 4. März 2026, von <https://md5decrypt.net/Api/>
- Mitnick, K. D., & Simon, W. L. (2001). *The art of deception: Controlling the human element of security*. John Wiley & Sons.
- Cybersecurity and Infrastructure Security Agency (CISA). (o. D.). *More than a password*. Abgerufen 20. März 2026, von <https://www.cisa.gov/MFA>
- National Institute of Standards and Technology (US). (2015). *Secure hash standard (SHS)*. (FIPS PUB 180-4). U.S. Department of Commerce. <https://nvl-pubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- Oechslin P. (2003). *Making a faster cryptanalytic time-memory trade-off*. In *Advances in Cryptology - CRYPTO 2003* (S. 617-630). Springer Berlin Heidelberg
- OWASP. (o. D.). *Password storage - OWASP cheat sheet series*. Abgerufen 20. März 2026, von https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Paar, C., & Pelzl, J. (2010). *Understanding cryptography: A textbook for students and practitioners*. Springer.

- Rivest, R. (1992). The MD5 Message-Digest Algorithm (RFC 1321). RFC Editor.
<https://www.rfc-editor.org/rfc/pdf/rfc1321.txt.pdf>
- Shannon, C. E. (1948). *A mathematical theory of communication*. The Bell System Technical Journal, 27(3), 379-423 <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>
- Sheldon, R. (2023, 10. Oktober). *What is password entropy?* TechTarget. Abgerufen am 17. März, von <https://www.techtarget.com/whatis/definition/password-entropy>
- Specops Software. (2025). *Passwort-Entropie als Maß für die Passwortsicherheit von Passwörtern und Passphrasen*. Abgerufen am 18 März 2026, von <https://specopssoft.com/de/blog/passwort-entropie-als-kriterium-fuer-passwort-sicherheit/>
- Wang, X., & Yu, H. (2005). How to break MD5 and other hash functions. In *Lecture Notes in Computer Science* (S. 19-35). Springer Berlin Heidelberg

6 Glossar

⊕ (XOR): Ein exklusives Oder, das eine logische Verknüpfung zweier Bits darstellt und genau dann einen Wert liefert, wenn die Eingabewerte unterschiedlich sind.

CPU: Central Processing Unit ist der Hauptprozessor eines Computers, der zentral-rechnerische Operationen und Steuerungsaufgaben übernimmt. Es ist nicht für parallelisierte Berechnungen optimiert, sondern für komplexe Einzelberechnungen.

GPU: Graphics Processing Unit ist der Grafikprozessor eines Computers, der aus Tausenden kleinen, parallel arbeitenden Rechenkernen besteht. Die GPU wurde darauf optimiert gleichzeitige Ausführungen vieler identischer Operationen, was sich in der Kryptologie zu einem bevorzugten Werkzeug macht

Klartext: Ist die unverschlüsselte oder ungehashte Version eines Passwortes bzw. der Ursprung einer Nachricht.

Modulo (mod): In der Informatik ist der Modulo ein Operator, der den Rest einer Division zweier Zahlen berechnet (in vielen Programmiersprachen als „%“ dargestellt).

Padding: Ist ein kryptographisches Verfahren, bei dem ein Klartext mit zusätzlichen Bits aufgefüllt wird (meistens mit einer 1 gefolgt von mehreren 0). Dies dient dazu, bei einem Hash Algorithmus die Blocklänge z.B. auf ein Vielfaches von 512 Bit zu bringen.

Passkey: ist ein moderner passwortloser Authentifizierungsstandard, der auf asymmetrische Kryptographie basiert. Anstatt ein Passwort einzutippen, authentifiziert sich der User über ein Smartphone, FaceID oder andere Möglichkeiten.

Postquantum Standard: bezieht sich auf neue Algorithmen, die so konstruiert sind, dass sie resistent gegen Quantencomputerangriffe sind.

Phishing: Ist eine Angriffsmethode, bei der Angreifer gefälschte E-Mail-Adressen oder Websites nutzen, um an sensible Daten von Opfern heranzugelang.

Register/Puffer: Sind interne Speicherbereiche innerhalb eines Prozessors oder eines Algorithmus, um Zwischenergebnisse zwischen Rechenrunden abzuspeichern oder zu manipulieren, bevor der endgültige Hash generiert wird.

ROTR (Rotate Right): Ist eine Operation, bei der Bits zyklisch nach rechts verschoben werden.

Secure Memory Zeroing: ist ein Sicherheitsverfahren, bei dem sensible Daten wie Passwörter im Arbeitsspeicher (RAM) nach der Verwendung sofort überschrieben und gelöscht werden, um zu verhindern, dass diese durch Angreifer ausgelesen werden.

Social Engineering: Ist die gezielte Manipulation oder die Anwendung psychologischer Tricks an Menschen, um an sensible Informationen oder Sicherheitsbarrieren zu gelangen.

Wörterbuchangriffe: Sind vorgefertigte Listen im Internet, die eine gigantische Menge an Klartextpasswörtern enthalten.

Zero-Day Exploits: sind Sicherheitslücken in der Software / Hardware, die dem Entwickler unbekannt sind. Angreifer können diese Schwachstellen unbemerkt ausnutzen, da Entwickler „Null Tage“ (zero-day) Zeit hatten, um ein Update zu entwickeln.

7 Anhang

Quellcode für die vorliegende Arbeit: https://github.com/dev-dawud/Facharbeit_Kryptologie_cpp

Tabellen:

Passwort	MD5-Zeit	SHA-256-Zeit
a	0.0002393 Sek.	0.0002201 Sek.
aa	0.0002314 Sek.	0.0011171 Sek.
aaa	0.0025297 Sek.	0.0516479 Sek.
aaaa	0.117373 Sek.	3.3166 Sek.
aaaaa	7.95039 Sek.	242.969 Sek. (ca. 4,05 Minuten)

Tabelle 1: Brute-Force Testergebnisse MD5 und SHA-256 - eigene Messung

Angriffstyp	Klartext Passwort	MD5	SHA-256
Rainbow-Table	test	0.076571 Sek.	1.77244 Sek.
Brute-Force	test	2.59766 Sek.	63.1237 Sek.

Tabelle 2: Vergleich Rainbow-Table und Brute-Force - eigene Messung

Angriffstyp	Klartext Passwort	MD5	SHA-256
Rainbow-Table	test	0.076571 Sek.	1.77244 Sek.
Rainbow-Table	t35!	6.80296 Sek.	144.477 Sek.
Brute-Force	test	3.03188 Sek.	69.189 Sek.
Brute-force	t35!	3.01423 Sek.	70.4321 Sek.
Rainbow-Table + salt	test	/	/
Rainbow-Table + salt + pepper	test	/	/
Brute-Force + salt	test	unvorstellbar	unvorstellbar
Brute-Force + salt + pepper	test	unvorstellbar	unvorstellbar

Tabelle 3: Stresstest salt und pepper - eigene Messung

Brute-Force Zeitverlauf nach Passwortlänge

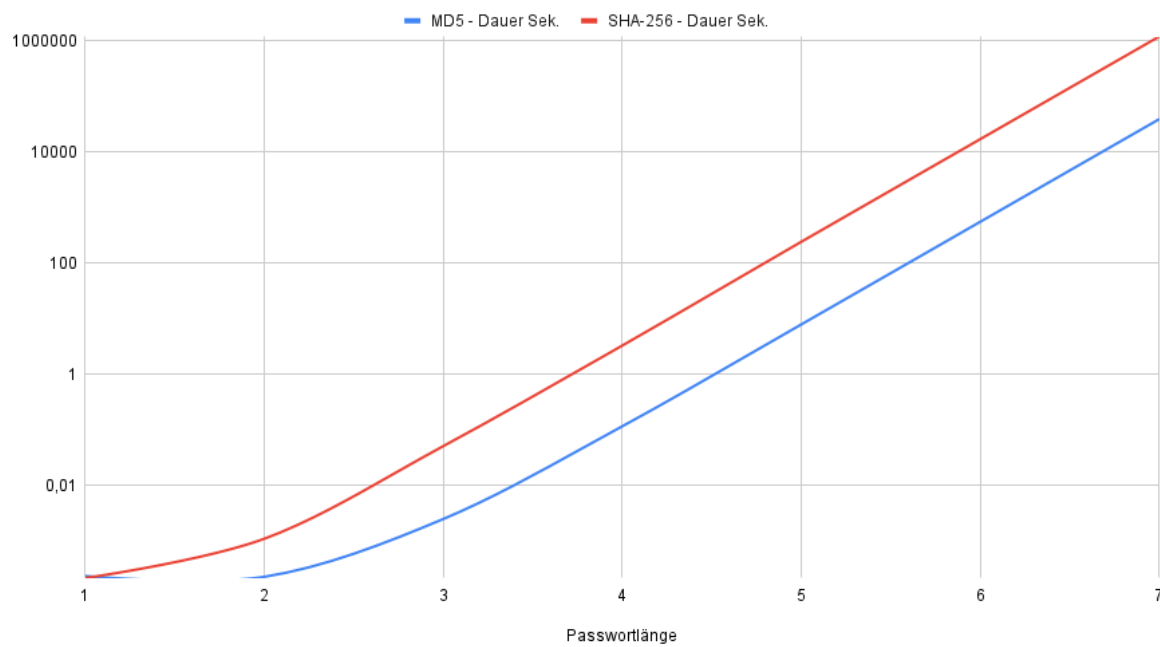


Abbildung 1: Zeitverlauf Brute-Force - eigene Messung – logarithmische Skala

8 Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich die vorliegende Facharbeit ohne fremde Hilfe verfasst und nur die im Literaturverzeichnis angegebenen Quellen verwendet habe.

Linz am 17.05.26 (Ort, Datum) David Wörb (Unterschrift)